Homework 2 – 2D-DCT

314552046 莊明憲

SPEC

- 2D-DCT
  - Implement 2D-DCT to transform “lena.png” to DCT coefficients (visualize in log domain).
    - Convert the input image to grayscale first.
    - Visualize the coefficients in the log domain. Feel free to scale and clip the coefficients for visualization.
  - Implement 2D-IDCT to reconstruct the image.
  - Evaluate the PSNR.
- Two 1D-DCT
  - Implement a fast algorithm by two 1D-DCT to transform “lena.png” to DCT coefficients.
- Compare the runtime between 2D-DCT and two 1D-DCT.
- Do **not** use any functions for DCT and IDCT, e.g., cv2.dct
  - Although, you can still use these functions to validate your output.
- Deadline: 2024/10/14 1:19 PM
- Upload to E3 with required files :
  - **VC_HW2_[student_id].pdf**: Report PDF
  - **VC_HW2_[student_id].zip**: Zipped source code (C/C++/Python/MATLAB) and a **README** file

Formula

Two-dimensional DCT & IDCT

$$F(u,v) = \frac{2}{N} C(u)C(v) \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x,y) \cos\frac{(2x+1)u\pi}{2N} \cos\frac{(2y+1)v\pi}{2N}$$

$$f(x,y) = \frac{2}{N} \sum_{u=0}^{N-1} \sum_{v=0}^{N-1} C(u)C(v)F(u,v) \cos\frac{(2x+1)u\pi}{2N} \cos\frac{(2y+1)v\pi}{2N}$$

$$\text{Where } C(u), C(v) = \begin{cases} \frac{1}{\sqrt{2}} & \text{for } u, v = 0 \\ 1 & \text{otherwise} \end{cases}$$

$$0 \le x, y, u, v \le N-1, N^2\text{: frame size}$$

One-dimensional DCT & IDCT

$$F(u,v) = \sqrt{\frac{2}{N}} C(u) \sum_{x=0}^{N-1} f(x) \cos \frac{(2x+1)u\pi}{2N}$$

$$f(x,y) = \sqrt{\frac{2}{N}} \sum_{u=0}^{N-1} C(u)F(u) \cos \frac{(2x+1)u\pi}{2N}$$

$$\text{Where } C(u) = \begin{cases} \frac{1}{\sqrt{2}} & \text{for } u = 0 \\ 1 & \text{otherwise} \end{cases}$$

$$0 \le x, u \le N-1, N^2\text{: frame size}$$

Code Explanation

1. 2D-DCT & Mathematical Helper function

The helper function C(k) defines the scaling factor used in the DCT computation.

The function dct_2d(f_xy) computes the 2D-DCT of the input image.
It first initializes the output array and generates index grids x and y.
Two nested loops iterate over each frequency pair (u, v).
Inside the loops, cosine terms for both directions are precomputed, and np.sum() is used to replace the double summation in the original formula for faster computation. The results are stored in F_uv, which holds the final DCT coefficients.

```python
# Mathematical Helper Function
def C(k):
    """
    Calculates the C(k) scaling factor used in DCT formulas.
    Returns 1/sqrt(2) for k=0, and 1 otherwise.
    """
    return 1 / math.sqrt(2) if k == 0 else 1

def dct_2d(f_xy):
    """
    Computes the 2D-DCT based on its formula.
    """
    M, N = f_xy.shape
    F_uv = np.zeros_like(f_xy, dtype=np.float64)

    x = np.arange(M).reshape(-1, 1)
    y = np.arange(N).reshape(1, -1)

    print(f"Calculating 2D-DCT ...")
    for u in range(M):
        for v in range(N):
            cos_u = np.cos((2 * x + 1) * u * math.pi / (2 * M))
            cos_v = np.cos((2 * y + 1) * v * math.pi / (2 * N))
            summation = np.sum(f_xy * cos_u * cos_v)
            scale_factor = (2 / N) * C(u) * C(v)
            F_uv[u, v] = scale_factor * summation

    return F_uv
```

2. 2D-IDCT

This function idct_2d(F_uv) performs the inverse 2D-DCT operation. It first initializes an empty array for the reconstructed image and prepares index vectors u and v. To simplify the scaling computation, two vectors c_u and c_v are defined and combined using np.outer() to form a 2D scaling matrix. During the nested loops over (x, y), cosine terms are generated similarly to the forward DCT. Inside each loop, np.sum() is used to sum over the entire matrix product scale_matrix * F_uv * cos_u * cos_v, effectively replacing the double summation in the formula with a single optimized numpy operation. The resulting value is stored in f_xy[x, y].

```python
def idct_2d(F_uv):
    """
    Computes the 2D-IDCT, corresponding to the idct_2d formulas.
    """
    M, N = F_uv.shape
    f_xy = np.zeros_like(F_uv, dtype=np.float64)

    u = np.arange(M).reshape(-1, 1)
    v = np.arange(N).reshape(1, -1)

    c_u = np.full(M, 1.0); c_u[0] = 1 / math.sqrt(2)
    c_v = np.full(N, 1.0); c_v[0] = 1 / math.sqrt(2)
    scale_matrix = np.outer(c_u, c_v)

    print(f"Calculating 2D-IDCT ...")
    for x in range(M):
        for y in range(N):
            cos_u = np.cos((2 * x + 1) * u * math.pi / (2 * M))
            cos_v = np.cos((2 * y + 1) * v * math.pi / (2 * N))
            summation = np.sum(scale_matrix * F_uv * cos_u * cos_v)
            f_xy[x, y] = (2 / N) * summation

    return f_xy
```

3. Two 1D-DCT

This function implements 2D-DCT by **calling 1D-DCT twice**. It first uses np.cos to build a cosine vector for each frequency term, allowing reuse of these precomputed values when computing the dot product with the input signal. The expression f_x @ cos_terms efficiently replaces manual summations and accelerates the transformation. The scaling factor adjusts the coefficient magnitude based on DCT normalization. Finally, np.apply_along_axis applies dct_1d along rows and then columns, completing the 2D transform and returning the coefficient matrix F_uv.

```python
def dct_two_1d(f_xy):
    """
    Computes the 2D-DCT by applying 1D-DCT function twice,
    """
    def dct_1d(f_x):
        N = len(f_x)
        F_u = np.zeros(N)
        x = np.arange(N)

        for u in range(N):
            cos_terms = np.cos((2 * x + 1) * u * math.pi / (2 * N))
            summation = f_x @ cos_terms
            scale_factor = math.sqrt(2 / N) * C(u)
            F_u[u] = scale_factor * summation
        return F_u

    print(f"Calculating Two 1D-DCT ...")
    intermediate = np.apply_along_axis(dct_1d, 1, f_xy)
    F_uv = np.apply_along_axis(dct_1d, 0, intermediate)

    return F_uv
```

4. Two 1D-IDCT

This function performs 2D-IDCT by applying **1D-IDCT twice**, reversing the 2D-DCT process. Inside idct_1d, a coefficient vector c_u is first initialized, with the DC term scaled by 1/sqrt(2) to match the normalization rule. Then np.cos builds cosine basis vectors for each spatial position, allowing reuse during the dot product computation c_u * F_u @ cos_terms, which efficiently replaces nested loops. The scaling factor sqrt(2/N) restores the original signal amplitude. Finally, np.apply_along_axis applies idct_1d along both axes—first rows, then columns—to reconstruct the spatial-domain image matrix f_xy from the frequency-domain coefficients.

```
def idct_two_1d(F_uv):
    """
    Computes the 2D-IDCT, corresponding to the dct_two_1d function.
    """
    def idct_1d(F_u):
        N = len(F_u)
        f_x = np.zeros(N)
        u = np.arange(N)

        c_u = np.full(N, 1.0); c_u[0] = 1 / math.sqrt(2)

        for x in range(N):
            cos_terms = np.cos((2 * x + 1) * u * math.pi / (2 * N))
            summation = (c_u * F_u) @ cos_terms
            scale_factor = math.sqrt(2 / N)
            f_x[x] = scale_factor * summation
        return f_x

    intermediate = np.apply_along_axis(idct_1d, 1, F_uv)
    f_xy = np.apply_along_axis(idct_1d, 0, intermediate)

    return f_xy
```

5. Get_visual_dct_image

This function adjusts the DCT coefficients for better visualization. I first apply a logarithmic scale to compress the large dynamic range, making both low- and high-frequency components more visible. Then, I clip the coefficient values between the 1st and 99th percentiles to remove extreme outliers that would otherwise dominate the contrast. Finally, I normalize the result to 8-bit intensity values for display, producing a visually balanced grayscale representation of the

DCT spectrum.

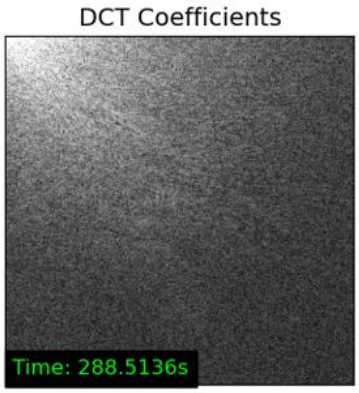
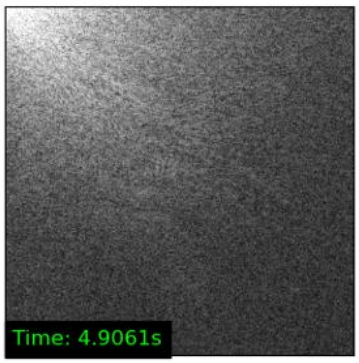
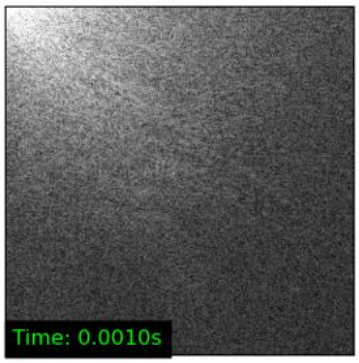
```python
def get_visual_dct_image(dct_coeffs):
    """Generates a visually enhanced 8-bit image from DCT coefficients for display."""
    log_coeffs = np.log1p(np.abs(dct_coeffs))
    ac_coeffs = log_coeffs[1:, 1:]
    vmin = np.min(ac_coeffs)
    vmax = np.percentile(ac_coeffs, 99)
    clipped_coeffs = np.clip(log_coeffs, vmin, vmax)
    scale = 255.0 / (vmax - vmin)
    visual_image = ((clipped_coeffs - vmin) * scale).astype(np.uint8)
    return visual_image
```

6. Main block

This section compares the performance of three different implementations. I measured the runtime for the standard 2D-DCT, the optimized version using two sequential 1D-DCTs, and OpenCV's built-in DCT/IDCT functions for validation. Each method also reconstructs the image using its corresponding IDCT and calculates PSNR to evaluate reconstruction quality. This comparison highlights the efficiency advantage of the two 1D-DCT approach while confirming correctness through the OpenCV results.

```python
def main():
    image_path = "lena.png"
    output_folder = "output_images"
    if not os.path.exists(output_folder): os.makedirs(output_folder)

    try:
        with Image.open(image_path) as img:
            original_img = np.array(img.convert('L'))
    except FileNotFoundError:
        print(f"Error: Image file '{image_path}' not found.")
        return

    f_xy = np.float64(original_img)
    print(f"Successfully loaded and converted image to grayscale. Size: {f_xy.shape[0]}x{f_xy.shape[1]}")
    plt.imsave(os.path.join(output_folder, "original_grayscale.png"), original_img, cmap='gray')
    print("-" * 30)

    # Method 1: 2D-DCT
    print("\nExecuting Method 1: 2D-DCT ...")
    start_time_2d = time.time()
    F_uv_2d = dct_2d(f_xy)
    time_2d = time.time() - start_time_2d
    reconstructed_f_xy_2d = idct_2d(F_uv_2d)
    psnr_2d = calculate_psnr(original_img, reconstructed_f_xy_2d)

    # Method 2: Two 1D-DCT
    print("\nExecuting Method 2: Two 1D-DCT ...")
    start_time_1d = time.time()
    F_uv_1d = dct_two_1d(f_xy)
    time_1d = time.time() - start_time_1d
    reconstructed_f_xy_1d = idct_two_1d(F_uv_1d)
    psnr_1d = calculate_psnr(original_img, reconstructed_f_xy_1d)

    # Method 3: OpenCV built-in functions for validation
    print("\nExecuting Method 3: OpenCV (for validation)...")
    start_time_cv2 = time.time()
    F_uv_cv2 = cv2.dct(f_xy.astype(np.float32))
    time_cv2 = time.time() - start_time_cv2
    reconstructed_f_xy_cv2 = cv2.idct(F_uv_cv2)
    psnr_cv2 = calculate_psnr(original_img, reconstructed_f_xy_cv2)
```

Result

	DCT Coefficient	Reconstructed Image
2D-DCT	My 2D-DCT 	
Two 1D-DCT	My Two 1D-DCT 	
OpenCV-DCT	OpenCV Validation 	

```
PS E:\Documents\課程內容\碩一上\視訊壓縮\HW2 - 2D-DCT> python VC_HW2_314552046.py
Successfully loaded and converted image to grayscale. Size: 512x512
-----------------------------

Executing Method 1: 2D-DCT ...
Calculating 2D-DCT ...
Calculating 2D-IDCT ...

Executing Method 2: Two 1D-DCT ...
Calculating Two 1D-DCT ...

Executing Method 3: OpenCV (for validation)...

======================== Final Comparison Report ========================
Method 1 (My 2D-DCT)      Execution time: 288.513579 sec, PSNR: 274.53 dB
Method 2 (My Two 1D-DCT)    Execution time: 4.906136 sec, PSNR: 274.53 dB
Method 3 (OpenCV Validation)      Execution time: 0.000998 sec, PSNR: 141.67 dB
=========================================================================
```

Comparison & Analysis

i. The result shows that the **Two 1D-DCT implementation** is dramatically faster than the direct 2D-DCT, reducing execution time from about 288 seconds to around 5 seconds — roughly **60× faster**. This confirms that decomposing 2D-DCT into two sequential 1D operations is a highly effective optimization.

ii. The **PSNR values** of both self-implemented methods are identical (274.53 dB), proving their correctness and numerical consistency.

iii. The **OpenCV implementation** runs almost instantly but produces a significantly lower PSNR (141.67 dB). This suggests that OpenCV's internal DCT likely uses different scaling or approximation strategies that trade accuracy for speed, making it less ideal for strict reconstruction but excellent for real-time applications.